Learning to Rank

Avishek Anand

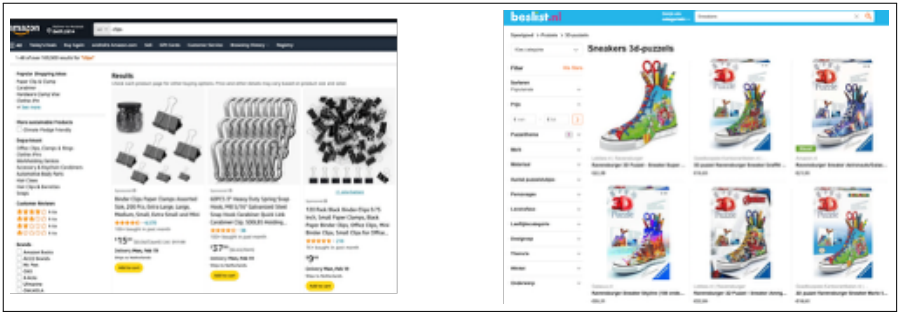
Outline

1. Motivation: Relevance Is Not King
2. Evaluation Metrics — What Are We Optimising?
3. Pointwise Approaches
4. Pairwise Approaches
5. Listwise Approaches
6. Summary: Choosing Your Paradigm

Roadmap

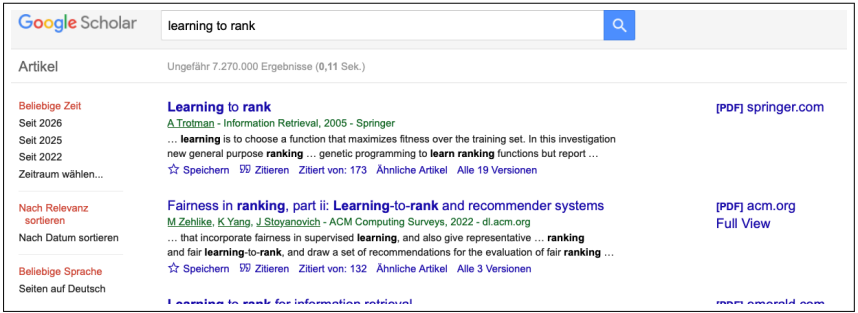
1. Motivation: Relevance Is Not King
2. Evaluation Metrics — What Are We Optimising?
3. Pointwise Approaches
4. Pairwise Approaches
5. Listwise Approaches
6. Summary: Choosing Your Paradigm

Scenario 1: Product Search (E-commerce)



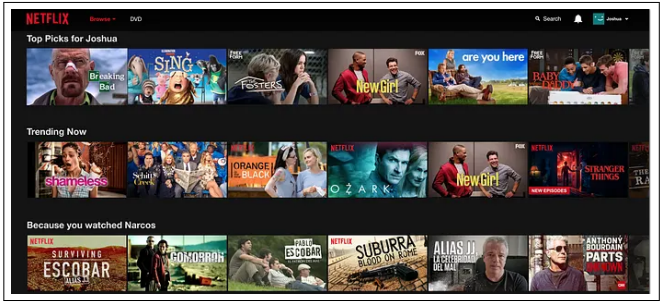
- **Objectives:** Relevance, Revenue / margin, Customer satisfaction, Sponsored constraints
- **Features:** Text match (BM25/embeddings), Price + discounts, Ratings / reviews, Shipping speed, stock, Seller quality
- **Feedback:** Clicks, add-to-cart, Purchases/returns, Position bias, Promo effects

Scenario 2: Academic Search (Scholar)



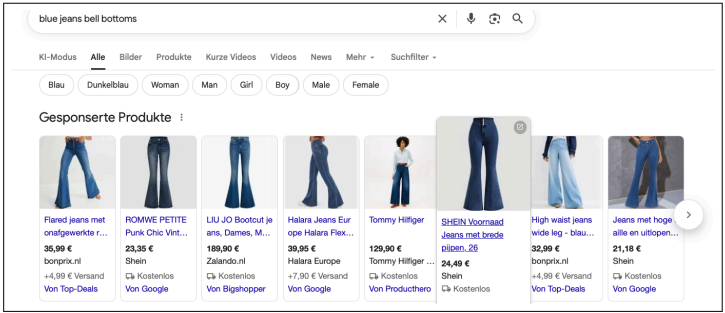
- **Objectives:** Relevance, Authority / impact, Recency (sometimes), Diversity of viewpoints
- **Features:** Text similarity, Citations, PageRank-like, Venue / author signals, Year, topicality
- **Feedback:** Clicks / saves, Citations (delayed), Popularity bias, Interface position bias

Scenario 3: Streaming (Netflix / Spotify)



- **Objectives:** Engagement (watch time), Retention (long-term), Novelty / diversity, Platform strategy
- **Features:** User/item embeddings, Recent interactions, Context (time/device), Similar-user patterns
- **Feedback:** Plays, skips, Watch completion, Dwell time is noisy, Exposure bias

Scenario 4: Online Advertising



- **Objectives:** Revenue, User experience, Advertiser value, Policy / fairness constraints
- **Features:** Bid, predicted CTR, Ad quality score, Query-ad relevance, User/context signals
- **Feedback:** Clicks, conversions, Attribution noise, Strategic bidding, Strong position bias

What Is Common Across All These Use Cases?

We keep seeing the same pattern.

What Is Common Across All These Use Cases?

We keep seeing the same pattern.

- **Large feature space**: 100s–1000s signals with strong interactions.
- **Multi-objective**: Balancing relevance with business goals and long-term value.
- **Noisy feedback**: Clicks are biased indicators; outcomes can be delayed.

What Is Common Across All These Use Cases?

We keep seeing the same pattern.

- **Large feature space**: 100s–1000s signals with strong interactions.
- **Multi-objective**: Balancing relevance with business goals and long-term value.
- **Noisy feedback**: Clicks are biased indicators; outcomes can be delayed.

So how do we learn a good ranking function?

A Document as a Feature Vector

Learning to Rank represents each query-document pair as a vector x.

$$\Phi(q, d) = \underbrace{\begin{array}{c|c|c|c|c} x_1 & x_2 & x_3 & \cdots & x_n \\ \text{BM25 Score} & \text{PageRank} & \text{Click Count} & & \text{Freshness} \end{array}}_{\mathbf{x}_{q,d} \in \mathbb{R}^{1000+}}$$

A Document as a Feature Vector

Learning to Rank represents each query-document pair as a vector x.

$$\Phi(q, d) = \underbrace{\begin{array}{c|c|c|c|c} x_1 & x_2 & x_3 & \cdots & x_n \\ \text{BM25 Score} & \text{PageRank} & \text{Click Count} & & \text{Freshness} \end{array}}_{\mathbf{x}_{q,d} \in \mathbb{R}^{1000+}}$$

The LTR Goal

Learn $f(\mathbf{x}) \rightarrow$ score

for optimal ranking

Why Learning to Rank?

Problem: We need a scoring function that combines many signals:

$$s(q, d) = f(x_{q,d})$$

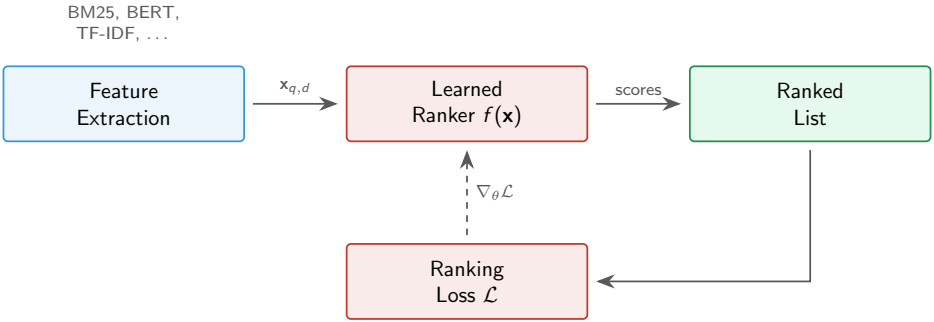
where $x_{q,d}$ can have hundreds of components.

- Hand-tuned rules do not scale (too many interactions).
- Objectives conflict (relevance vs revenue vs engagement).
- Supervision is weak and biased (implicit feedback).

Learning to Rank is the engineering solution.

How LTR Models Learn

Training an LTR model is a closed-loop iterative process.



- **Loss** ($\mathcal{L}$) measures the "badness" of the current ranked list.
- **Gradient** ($\nabla_\theta \mathcal{L}$) tells us how to nudge θ to improve the metric.

Roadmap

1. Motivation: Relevance Is Not King
2. Evaluation Metrics — What Are We Optimising?
3. Pointwise Approaches
4. Pairwise Approaches
5. Listwise Approaches
6. Summary: Choosing Your Paradigm

Section 2

Evaluation Metrics — What Are We Optimising?

Recap: Binary Metrics

Traditional metrics focus on whether a document is "relevant" or not.

- **P@**k **/ R@**k: Good for broad recall but position-blind within top-k.
- **MRR**: Excellent for navigational/QA tasks (cares about first result).
- **MAP**: Averages precision across all relevant docs; sensitive to rank.

Recap: Binary Metrics

Traditional metrics focus on whether a document is "relevant" or not.

- **P@**k **/ R@**k: Good for broad recall but position-blind within top-k.
- **MRR**: Excellent for navigational/QA tasks (cares about first result).
- **MAP**: Averages precision across all relevant docs; sensitive to rank.

Recap: Binary Metrics

Traditional metrics focus on whether a document is "relevant" or not.

- **P@k / R@k**: Good for broad recall but position-blind within top-k.
- **MRR**: Excellent for navigational/QA tasks (cares about first result).
- **MAP**: Averages precision across all relevant docs; sensitive to rank.

The Limitation

Binary metrics ignore the **intensity** of relevance. In modern search, we often have graded labels (e.g., Perfect, Good, Fair, Bad).

NDCG: The Gold Standard for LTR

NDCG handles graded relevance and prioritizes top-heavy rankings.

- **Grades**: Cumulative Gain (CG) sums relevance scores: $\sum \text{rel}_i$.
- **Discount**: DCG applies a logarithmic decay: $\sum \frac{\text{rel}_i}{\log_2(i+1)}$.
- **Normalization**: NDCG scales DCG to $[0, 1]$ relative to the ideal sort.

NDCG: The Gold Standard for LTR

NDCG handles graded relevance and prioritizes top-heavy rankings.

- **Grades**: Cumulative Gain (CG) sums relevance scores: $\sum \text{rel}_i$.
- **Discount**: DCG applies a logarithmic decay: $\sum \frac{\text{rel}_i}{\log_2(i+1)}$.
- **Normalization**: NDCG scales DCG to $[0, 1]$ relative to the ideal sort.

NDCG: The Gold Standard for LTR

NDCG handles graded relevance and prioritizes top-heavy rankings.

- **Grades**: Cumulative Gain (CG) sums relevance scores: $\sum \text{rel}_i$.
- **Discount**: DCG applies a logarithmic decay: $\sum \frac{\text{rel}_i}{\log_2(i+1)}$.
- **Normalization**: NDCG scales DCG to $[0, 1]$ relative to the ideal sort.

$$\text{NDCG@}k = \frac{\sum_{i=1}^{k} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}}{\text{IDCG@}k}$$

Worked Example: Computing NDCG@5

Let's calculate the cost of a "miss" in the top positions.

- **Current**: Grades $[2, 3, 1, 0, 0] \rightarrow \text{DCG} \approx 4.39$.
- **Ideal**: Grades $[3, 2, 1, 0, 0] \rightarrow \text{DCG} \approx 4.76$.
- **Result**: $\text{NDCG} = 4.39/4.76 \approx \mathbf{0.92}$.

Worked Example: Computing NDCG@5

Let's calculate the cost of a "miss" in the top positions.

- **Current**: Grades $[2, 3, 1, 0, 0] \rightarrow \text{DCG} \approx 4.39$.
- **Ideal**: Grades $[3, 2, 1, 0, 0] \rightarrow \text{DCG} \approx 4.76$.
- **Result**: $\text{NDCG} = 4.39/4.76 \approx \mathbf{0.92}$.

Worked Example: Computing NDCG@5

Let's calculate the cost of a "miss" in the top positions.

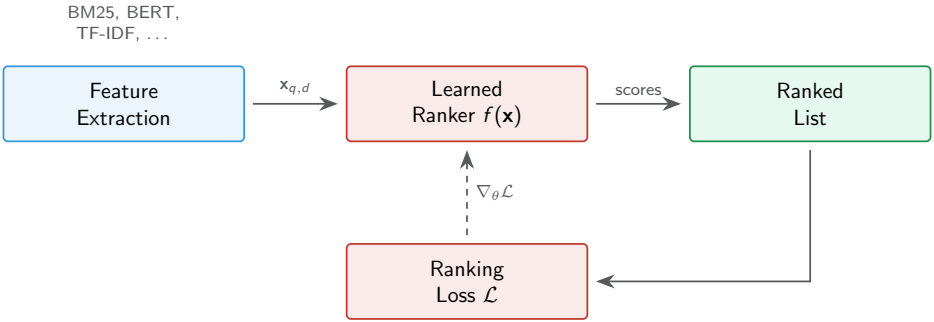
- **Current**: Grades $[2, 3, 1, 0, 0] \rightarrow \text{DCG} \approx 4.39$.
- **Ideal**: Grades $[3, 2, 1, 0, 0] \rightarrow \text{DCG} \approx 4.76$.
- **Result**: $\text{NDCG} = 4.39/4.76 \approx \mathbf{0.92}$.

The Logarithmic Effect

Notice that moving the "3" from position 1 to 2 caused an 8% drop in score. NDCG is extremely sensitive to errors at the very top.

How LTR Models Learn

Training an LTR model is a closed-loop iterative process.



- **Loss** ($\mathcal{L}$) measures the "badness" of the current ranked list.
- **Gradient** ($\nabla_\theta \mathcal{L}$) tells us how to nudge θ to improve the metric.

The Main Challenge for LTR

The sorting function (or the rank of a document) is difficult to optimize directly.

Ranking metrics (NDCG, MAP, etc.) as functions of document scores are:

- **Non-smooth**
- **Mostly flat**
- **Discontinuous**

The Derivative Problem

Small score changes $\Rightarrow$ no metric change.
Tiny score swap $\Rightarrow$ big metric jump.

Gradients of the true metric are unusable. Each family of LTR methods has its own approach to overcoming this problem.

Comparing LTR Paradigms

The primary difference lies in the unit of training and the loss focus.

Paradigm	Training Instance	Model Learns
Pointwise	One query-doc pair	Absolute relevance score
Pairwise	One ordered pair (d_i, d_j)	Relative preference
Listwise	Full query candidate set	Quality of full ranking

In this lecture, we shift focus to the most powerful of these: the Listwise approach.

Roadmap

1. Motivation: Relevance Is Not King
2. Evaluation Metrics — What Are We Optimising?
3. Pointwise Approaches
4. Pairwise Approaches
5. Listwise Approaches
6. Summary: Choosing Your Paradigm

Section 3

Pointwise Approaches

1. Where Do Labels Come From?

To train a model, we need target labels y for query-document pairs.

- **Explicit Ratings**: Human judges grade relevance (e.g., 0=Bad to 4=Perfect). Expensive but high quality.
- **Implicit Feedback**: User behavior parsed from logs.
  - **Clicks**: Did the user click? (Binary)
  - **Dwell Time**: How long did they stay?
  - **Skips**: Did they scroll past it?

2. The Training Table

We flatten the data into a standard machine learning dataset.

Each row is an isolated instance: a Query q, a Document d, their extracted features $\mathbf{x}$, and the target label y.

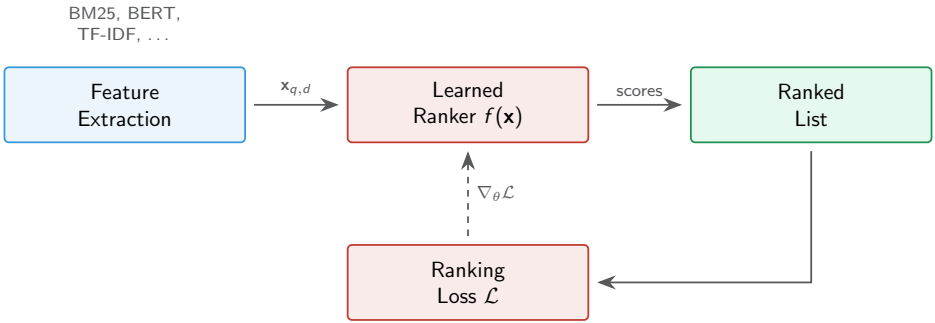
Query ID	Doc ID	BM25 (x_1)	PageRank (x_2)	...	Label (y)
q_1	d_A	12.4	0.85	...	**1 (Click)**
q_1	d_B	8.1	0.92	...	**0 (Skip)**
q_1	d_C	15.2	0.12	...	**1 (Click)**
q_2	d_X	5.5	0.44	...	**0 (Skip)**

Key Insight

The model sees rows independently. It does not know that d_A, d_B, d_C belong to the same query q_1 during the forward pass.

Which Model do we use?

Training an LTR model is a closed-loop iterative process.



- **Loss** ($\mathcal{L}$) measures the "badness" of the current ranked list.
- **Gradient** ($\nabla_\theta \mathcal{L}$) tells us how to nudge θ to improve the metric.

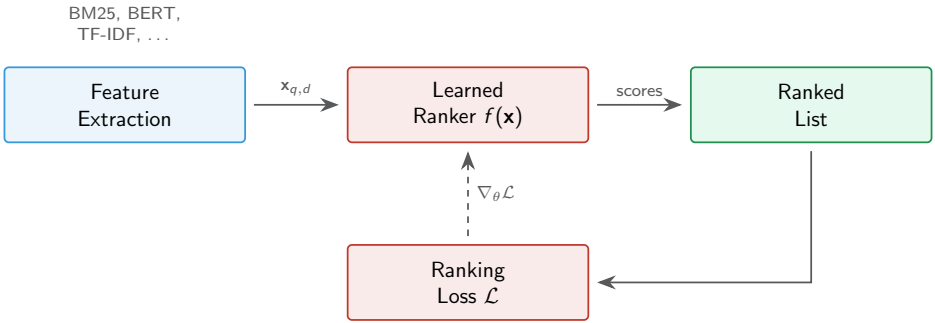
3. Mapping Features to a Score

A Pointwise model $f(\mathbf{x})$ maps the feature vector to a predicted score $\hat{y}$.

- **Linear Model**: $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$. (Fast, but assumes linear relationships).
- **Neural Network**: $\hat{y} = \text{MLP}(\mathbf{x})$. (Captures complex feature interactions).
- **Tree Ensemble**: $\hat{y} = \sum \text{Tree}(\mathbf{x})$. (Handles heterogeneous features well).

Which Loss do we use?

Training an LTR model is a closed-loop iterative process.



- **Loss** ($\mathcal{L}$) measures the "badness" of the current ranked list.
- **Gradient** ($\nabla_\theta \mathcal{L}$) tells us how to nudge θ to improve the metric.

4. The Pointwise Loss per Example

We train the model to minimize the error between prediction $\hat{y}$ and label y.

- **Regression** (e.g., MSE): $\mathcal{L} = (y - \hat{y})^2$

4. The Pointwise Loss per Example

We train the model to minimize the error between prediction $\hat{y}$ and label y.

- **Regression** (e.g., MSE): $\mathcal{L} = (y - \hat{y})^2$
- **Classification** (e.g., BCE): $\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$

4. The Pointwise Loss per Example

We train the model to minimize the error between prediction $\hat{y}$ and label y.

- **Regression** (e.g., MSE): $\mathcal{L} = (y - \hat{y})^2$
- **Classification** (e.g., BCE): $\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$

What is it actually doing?

For q_1, d_A, the loss penalizes the model if $\hat{y}$ is far from 1. It updates the weights θ to make $\hat{y}$ closer to 1 **in isolation**.

Limitation of Pointwise Methods

Pointwise losses do not directly optimize ranking quality. A lower loss does not mean a better ranking!

Model A

Rel.	Score	Rank
1	0.6	1
0	0.5	2
0	0.5	3
0	0.5	4
0	0.5	5

Model B

Rel.	Score	Rank
1	0.1	5
0	0.2	1
0	0.2	2
0	0.2	3
0	0.2	4

Mismatch: Ranking is fundamentally about the **relative order** of documents, not fitting a curve.

Pointwise in Disguise: Standard Cross-Encoders

Recall monoBERT from Lecture 4 is technically a pointwise ranker.

- **Input**: Joint query-document sequence `[CLS] Q [SEP] D`.
- **Loss**: Binary Cross-Entropy on the `[CLS]` representation.
- **Success**: Works well because BERT captures deep semantics, but **not** because the loss is optimal for ranking.

Pointwise in Disguise: Standard Cross-Encoders

Recall monoBERT from Lecture 4 is technically a pointwise ranker.

- **Input**: Joint query-document sequence `[CLS] Q [SEP] D`.
- **Loss**: Binary Cross-Entropy on the `[CLS]` representation.
- **Success**: Works well because BERT captures deep semantics, but **not** because the loss is optimal for ranking.

Pointwise in Disguise: Standard Cross-Encoders

Recall monoBERT from Lecture 4 is technically a pointwise ranker.

- **Input**: Joint query-document sequence `[CLS] Q [SEP] D`.
- **Loss**: Binary Cross-Entropy on the `[CLS]` representation.
- **Success**: Works well because BERT captures deep semantics, but **not** because the loss is optimal for ranking.

The Capacity Paradox

With enough data and model capacity (like LLMs), simplistic pointwise losses often reach surprisingly high performance.

Roadmap

1. Motivation: Relevance Is Not King
2. Evaluation Metrics — What Are We Optimising?
3. Pointwise Approaches
4. Pairwise Approaches
5. Listwise Approaches
6. Summary: Choosing Your Paradigm

Section 4

Pairwise Approaches

Pairwise: Relative Preferences

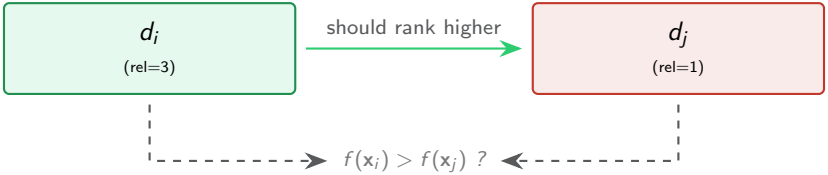
Instead of absolute scores, we learn which document is "better" in a pair.

- **Intuition**: For each pair (d_i, d_j), if $y_i > y_j$, the model should score $s_i > s_j$.
- **Strength**: Transforms ranking into binary classification of pairs.
- **Feedback**: Errors are symmetric — push one up, the other down.

Pairwise: Relative Preferences

Instead of absolute scores, we learn which document is "better" in a pair.

- **Intuition**: For each pair (d_i, d_j), if $y_i > y_j$, the model should score $s_i > s_j$.
- **Strength**: Transforms ranking into binary classification of pairs.
- **Feedback**: Errors are symmetric — push one up, the other down.



Goal: Maximize agreement with labels

RankNet: Learning from Pairs

Pairwise loss minimizes the average number of pair-inversions in rankings.

For all pairs where $y_i > y_j$:

$$\mathcal{L}_{pair} = \sum_{y_i > y_j} \phi(s_i - s_j)$$

RankNet is the most prominent pairwise method:

$$\mathcal{L}_{RankNet} = \sum_{y_i > y_j} \log(1 + e^{-(s_i - s_j)})$$

Burges, C. et al. Learning to rank using gradient descent. ICML 2005.

Limitation of Pairwise Methods

We minimize incorrect inversions. However, not every pair is equally important.

Ranking 1

Rel.	Score	Rank
1	0.6	1
0	0.2	4
1	0.1	5
0	0.4	3
0	0.8	2

Pairs correct: 9

Ranking 2

Rel.	Score	Rank
0	0.8	1
0	0.6	2
1	0.4	3
0	0.2	4
1	0.1	5

Pairs correct: 10

Mismatch: It's much more important to get the correct ordering of top documents than bottom ones. Pairwise treats all pairs equally.

Worked Example: RankNet Misstep

Suppose a Blog (rel=1) is scored at 2.5, while a Tier-1 list (rel=3) is at 2.1.

- **Conflict**: $P(\text{Tier-1} \succ \text{Blog}) \approx 0.40$ (Model is wrong).
- **Gradient**: Pushes Tier-1 score up ($+0.60$) and Blog down (-0.60).
- **Observation**: RankNet weights all swaps equally, regardless of position.

Worked Example: RankNet Misstep

Suppose a Blog (rel=1) is scored at 2.5, while a Tier-1 list (rel=3) is at 2.1.

- **Conflict**: $P(\text{Tier-1} \succ \text{Blog}) \approx 0.40$ (Model is wrong).
- **Gradient**: Pushes Tier-1 score up ($+0.60$) and Blog down (-0.60).
- **Observation**: RankNet weights all swaps equally, regardless of position.

Worked Example: RankNet Misstep

Suppose a Blog (rel=1) is scored at 2.5, while a Tier-1 list (rel=3) is at 2.1.

- **Conflict**: $P(\text{Tier-1} \succ \text{Blog}) \approx 0.40$ (Model is wrong).
- **Gradient**: Pushes Tier-1 score up ($+0.60$) and Blog down (-0.60).
- **Observation**: RankNet weights all swaps equally, regardless of position.

The Position Blindness

Swapping docs at positions 1–2 should be "more important" than positions 99–100. RankNet's loss doesn't know about positions.

RankSVM: Margin-Based Ranking

Historical significance: First to use implicit feedback (clicks) for LTR.

- **Margin**: Force $\mathbf{w}^\top(\mathbf{x}_i - \mathbf{x}_j) \geq 1$ for correctly ordered pairs.
- **Feed**: Users clicking result i after skipping $j \implies d_i \succ d_j$.
- **Legacy**: Solid theory (max-margin) but less scalable than boosting.

RankSVM: Margin-Based Ranking

Historical significance: First to use implicit feedback (clicks) for LTR.

- **Margin**: Force $\mathbf{w}^\top(\mathbf{x}_i - \mathbf{x}_j) \geq 1$ for correctly ordered pairs.
- **Feed**: Users clicking result i after skipping $j \implies d_i \succ d_j$.
- **Legacy**: Solid theory (max-margin) but less scalable than boosting.

RankSVM: Margin-Based Ranking

Historical significance: First to use implicit feedback (clicks) for LTR.

- **Margin**: Force $\mathbf{w}^\top(\mathbf{x}_i - \mathbf{x}_j) \geq 1$ for correctly ordered pairs.
- **Feed**: Users clicking result i after skipping $j \implies d_i \succ d_j$.
- **Legacy**: Solid theory (max-margin) but less scalable than boosting.

Key Contribution

Joachims (2002) showed that even noisy implicit signals (clicks) can train a competitive ranker when framed as a pairwise problem.

Roadmap

1. Motivation: Relevance Is Not King
2. Evaluation Metrics — What Are We Optimising?
3. Pointwise Approaches
4. Pairwise Approaches
5. Listwise Approaches
6. Summary: Choosing Your Paradigm

Section 5

Listwise Approaches

Listwise: Learning from the Whole List

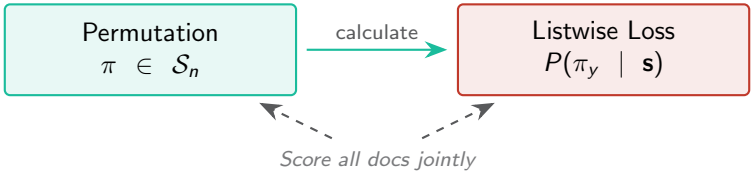
Listwise models treat the entire set of documents for a query as a single training unit.

- **Philosophy**: Don't look at doc pairs in isolation; look at the *permutation*.
- **Unit**: The model forward pass and loss are computed over all candidates simultaneously.
- **Structure**: Training data is explicitly grouped by query.

Listwise: Learning from the Whole List

Listwise models treat the entire set of documents for a query as a single training unit.

- **Philosophy**: Don't look at doc pairs in isolation; look at the *permutation*.
- **Unit**: The model forward pass and loss are computed over all candidates simultaneously.
- **Structure**: Training data is explicitly grouped by query.



Goal: Minimize distance between predicted and ideal permutations

Training Data for Listwise Ranking

Unlike pointwise rows, listwise data preserves the query-level grouping.

For one query q, we process the candidates as a set:

$$\{(d_1, \mathbf{x}_1, y_1), (d_2, \mathbf{x}_2, y_2), \ldots, (d_n, \mathbf{x}_n, y_n)\}$$

1. **Scoring**: Model computes scores $s_i = f_\theta(\mathbf{x}_i)$ for all d_i.
2. **Grouping**: Scores are gathered into a vector $\mathbf{s}_q$.
3. **Loss**: A single loss $\mathcal{L}(\mathbf{s}_q, \mathbf{y}_q)$ is calculated over the entire group.

This allows the model to "see" the competition between documents.

ListNet: Matching Probability Distributions

ListNet converts scores into a probability distribution and matches it to the target.

1. **Soft-max Distribution**: Convert scores s_i and labels y_i into probabilities:

$$P_s(d_i) = \frac{e^{s_i}}{\sum_j e^{s_j}}, \qquad P_y(d_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

2. **Cross-Entropy Loss**: Minimize the divergence between prediction and target:

$$\mathcal{L} = -\sum_i P_y(d_i) \log P_s(d_i)$$

- Focuses on matching the "Top-1" probability distribution.
- Fully differentiable; easy to train with standard SGD/Adam.

ListMLE: Plackett-Luce Likelihood

ListMLE maximizes the likelihood of observing the ground-truth permutation.

The probability of a permutation π given scores s is the product of sequential choices:

$$P(\pi \mid s) = \prod_{k=1}^{n} \frac{e^{s_{\pi(k)}}}{\sum_{j=k}^{n} e^{s_{\pi(j)}}}$$

- **Loss**: Negative log-likelihood $-\log P(\pi^* \mid s)$.
- **Intuition**: The model is trained so that the correct order is the most likely outcome of a sequential sampling process.
- The training object is the **entire ranking** simultaneously.

Limitation of Early Listwise Surrogates

ListNet and ListMLE optimize smooth proxies, not the actual ranking metrics.

The Metric GAP

Likelihood maximization and cross-entropy are "surrogate" losses.

- **Discordance**: Improving surrogate loss does not always improve NDCG or MAP.
- **Uniformity**: These losses often treat all positions as equally important in the probability space.

To fix this, we need "metric-aware" listwise optimization.

Roadmap

1. Motivation: Relevance Is Not King
2. Evaluation Metrics — What Are We Optimising?
3. Pointwise Approaches
4. Pairwise Approaches
5. Listwise Approaches
6. Summary: Choosing Your Paradigm

Section 6

Summary: Choosing Your Paradigm

Summary: Choosing Your Paradigm

Choose the paradigm that matches your data and model architecture.

- **Pointwise**: Simplest training, scalable, but weak ranking alignment.
- **Pairwise**: Models relative preferences; better for order than pointwise.
- **Listwise**: Aligns training with query-level ranking behavior.
- **LambdaMART**: Most important practical metric-aware method in industry.
- **Neural Listwise**: Best when learning representations jointly from text.

Summary: Choosing Your Paradigm

Choose the paradigm that matches your data and model architecture.

- **Pointwise**: Simplest training, scalable, but weak ranking alignment.
- **Pairwise**: Models relative preferences; better for order than pointwise.
- **Listwise**: Aligns training with query-level ranking behavior.
- **LambdaMART**: Most important practical metric-aware method in industry.
- **Neural Listwise**: Best when learning representations jointly from text.

Summary: Choosing Your Paradigm

Choose the paradigm that matches your data and model architecture.

- **Pointwise**: Simplest training, scalable, but weak ranking alignment.
- **Pairwise**: Models relative preferences; better for order than pointwise.
- **Listwise**: Aligns training with query-level ranking behavior.
- **LambdaMART**: Most important practical metric-aware method in industry.
- **Neural Listwise**: Best when learning representations jointly from text.

The Recall Ceiling

Remember: A perfect re-ranker cannot fix what the first stage missed. LTR is most effective when the retrieval stage has high recall.

Next Class: Evaluation & Large Scale Systems

Next week, we move into the "wild" — how to evaluate systems with real users.

- **Offline**: Benchmarks like TREC and the limits of static labels.
- **Online**: A/B testing, Interleaving, and Click-through analysis.
- **Deployment**: Building low-latency production pipelines.

Next Class: Evaluation & Large Scale Systems

Next week, we move into the "wild" — how to evaluate systems with real users.

- **Offline**: Benchmarks like TREC and the limits of static labels.
- **Online**: A/B testing, Interleaving, and Click-through analysis.
- **Deployment**: Building low-latency production pipelines.

Next Class: Evaluation & Large Scale Systems

Next week, we move into the "wild" — how to evaluate systems with real users.

- **Offline**: Benchmarks like TREC and the limits of static labels.
- **Online**: A/B testing, Interleaving, and Click-through analysis.
- **Deployment**: Building low-latency production pipelines.

Preparation

Review the PyTerrier documentation for the Learning to Rank module.